

Exercice 01 — TEST-1 a TEST-4 : environnement et premiers tests de la plateforme Code un max

Projet fil rouge : Code un max - Suite de tests automatisés de la plateforme de cours

C'est le **premier exercice** du projet fil rouge ! Vous allez poser les fondations du projet. Code un max souhaite fiabiliser sa plateforme web de cours en ligne. Au fil du module, les apprenants construisent progressivement une suite de tests automatisés complète : premiers tests unitaires et E2E, refactoring en Page Object Model, gestion des données avec Faker, mocking des services externes, puis intégration dans un pipeline GitHub Actions déclenché à chaque commit. Le projet couvre les parcours clés de la plateforme (login, catalogue de cours, inscription, tableau de bord) jusqu'à la mise en situation MSPR finale : application testée, suite complète, pipeline CI/CD et rapport pytest-html livrés et documentés.

Dans cet exercice : Code un max héberge une plateforme web de cours en ligne (login, catalogue, inscription, tableau de bord). Aucun test automatisé n'existe : chaque mise en production repose sur des vérifications manuelles. Tu es recruté pour poser les fondations de la suite de tests automatisée. Ce chapitre installe le socle sur lequel les chapitres suivants ajouteront le Page Object Model, les données Faker et la CI.

Avant de commencer

L'**Exemple 01** ([exemples/exemple-01/exemple-01.md](#)) t'a fait pratiquer la technique sur un cas neutre. Ici, tu l'appliques au projet **Code un max - Suite de tests automatisés de la plateforme de cours**. Assure-toi de l'avoir lu et suivi avant de commencer.

Objectif

Installer l'environnement de test, structurer le dépôt, lancer l'application de cours en local, et écrire les tout premiers tests (un unitaire, un E2E) sur la plateforme Code un max.

Prérequis

- Python 3.11+ installé
- Git
- VS Code
- Chrome ou Firefox
- Compte GitHub
- Bases de la programmation Python
- Notions de développement web

Scénario

L'application de demo t'est fournie (dossier app/ avec un petit serveur web servant les pages login et catalogue, plus un module de logique). Ta mission : rendre le projet testable, prouver que tout tourne en local, et livrer les deux premiers tests verts sous le ticket TEST-1 a TEST-4.

Étapes à réaliser

Étape 1 — Préparer le poste et le venv

Vérifie que Python 3.11+, Git et Chrome (ou Firefox) sont installés, puis crée un environnement virtuel dédié au projet.

Ce que vous devez faire :

- Vérifie les versions de python, git et du navigateur.
- Crée un venv à la racine du dépôt et active-le.
- Installe pytest et selenium, fige les versions dans un fichier requirements.txt.

Résultat attendu : Un venv actif, pytest et selenium installés, requirements.txt versionné.

Piste : `pip freeze > requirements.txt` capture exactement les versions installées.

Étape 2 — Structurer le dépôt de tests

Mets en place l'arborescence standard qui accueillera toute la suite au fil du module.

Ce que vous devez faire :

- Crée les dossiers tests/unit, tests/integration, tests/e2e.
- Ajoute un `conftest.py` à la racine des tests et un `pytest.ini` à la racine du projet.
- Dans `pytest.ini`, déclare les chemins de tests et des markers (unit, e2e).

Résultat attendu : `pytest --collect-only` trouve les dossiers sans erreur, même s'il n'y a pas encore de test.

Référence : Documentation pytest : configuration via `pytest.ini`, section `testpaths` et `markers`.

Erreur fréquente : Oublier les fichiers `init.py` ou mal configurer `testpaths`, ce qui fait que pytest ne collecte rien.

Étape 3 — Lancer la plateforme en local

Démarré l'application de cours fournie et confirme qu'elle répond sur le navigateur.

Ce que vous devez faire :

- Lis le README de `app/` pour identifier la commande de lancement et le port.
- Démarré le serveur et ouvre la page de login dans le navigateur.
- Note l'URL de base (ex: `http://localhost:8000`) : elle servira aux tests E2E.

Résultat attendu : La page de login de la plateforme s'affiche dans le navigateur à l'URL locale.

Piste : Garde le serveur dans un terminal à part : les tests E2E ont besoin qu'il tourne pendant leur exécution.

Étape 4 — TEST-1 : premier test unitaire sur la logique métier

Écris un test unitaire sur une fonction pure de l'application (par exemple la validation d'un email d'inscription ou le calcul de progression d'un cours).

Ce que vous devez faire :

- Repere une fonction sans dependance reseau dans app/.
- Ecris au moins deux cas dans tests/unit : un cas valide et un cas invalide.
- Marque le test avec @pytest.mark.unit.

Résultat attendu : pytest -v -m unit affiche les tests unitaires en PASSED.

Erreur fréquente : Choisir une fonction qui touche la base ou le reseau : ce n'est plus un test unitaire, il devient lent et fragile.

Étape 5 — TEST-2 : configurer le WebDriver headless

Prepare une fixture pytest qui fournit un navigateur Chrome (ou Firefox) headless a tes tests E2E.

Ce que vous devez faire :

- Dans conftest.py, cree une fixture driver qui instancie le WebDriver en headless.
- Assure le nettoyage du navigateur apres chaque test (quit).
- Rends le mode headless activable ou desactivable pour deboguer en local.

Résultat attendu : Un test bidon qui ouvre driver.get('URL de base') passe sans laisser de processus Chrome zombie.

Piste : Une variable d'environnement HEADLESS lue dans la fixture permet de basculer le mode sans modifier le code.

Étape 6 — TEST-3 : ouvrir la page de login et verifier le titre

Premier test E2E : la plateforme repond et affiche la bonne page.

Ce que vous devez faire :

- Utilise la fixture driver pour ouvrir la page de login de la plateforme.
- Verifie le titre de l'onglet ou la presence d'un element cle (champ email, bouton connexion).
- Marque le test avec @pytest.mark.e2e.

Résultat attendu : pytest -v -m e2e montre le test de login en PASSED, serveur lance en parallele.

Erreur fréquente : Lancer les tests E2E sans avoir demarre le serveur : le driver recoit une page d'erreur de connexion refusee.

Étape 7 — TEST-4 : localiser et verifier un element du catalogue

Deuxieme test E2E, sur un parcours different : le catalogue de cours doit afficher au moins un cours.

Ce que vous devez faire :

- Navigue vers la page catalogue depuis l'URL de base.
- Localise les elements de cours (par id, classe ou selecteur CSS).
- Verifie qu'au moins un cours est present et que son titre n'est pas vide.

Résultat attendu : Le test catalogue passe et prouve que Selenium sait lire du contenu dynamique de la plateforme.

Piste : `find_elements` (au pluriel) renvoie une liste : vérifie sa longueur avant d'accéder au premier élément.

Vérification

Quand vous avez terminé, vérifiez que :

- ☐ `pytest -v` lance les 4 tests (2 unitaires ou plus, 2 E2E) tous verts, serveur démarre.
 - ☐ `pytest -m unit` et `pytest -m e2e` filtrent correctement grâce aux markers.
 - ☐ `requirements.txt`, `pytest.ini` et `conftest.py` sont versionnés dans le dépôt.
 - ☐ Aucun processus navigateur ne reste ouvert après la fin des tests.
-

En pratique

Beaucoup démarrent en écrivant dix tests E2E et zéro unitaire parce que 'c'est plus concret'. Résultat : au premier changement de bouton, tout casse et personne ne sait pourquoi. Force-toi dès ce chapitre à mettre au moins un test unitaire : c'est lui qui te dira où est vraiment le bug quand l'E2E rougira.

Bonus (Facultatif mais Recommandé)

Ajoute un test E2E qui vérifie qu'un login avec un mauvais mot de passe affiche un message d'erreur sur la plateforme.

Livrable attendu

Un dépôt Git avec l'arborescence `tests/unit`, `tests/integration`, `tests/e2e`, `conftest.py`, `pytest.ini`, `requirements.txt`, et les 4 premiers tests (TEST-1 à TEST-4) qui passent en local.

Une fois terminé, consultez la **correction** dans [corrections-exercices/correction-exercice-01/correction-exercice-01.md](#) pour comparer votre travail.